

PERSONALIZED FITNESS CLASS BOOKING PLATFORM

Complete MERN Stack Web Application

Project Report

Technology

MongoDB • Express.js • React.js • Node.js

Project Type

Full-Stack Web Application

Prepared for HCL / GUVI Project Submission

Academic / Course Project Documentation

PROJECT DECLARATION

This project report presents the design and implementation plan for a Personalized Fitness Class Booking Platform developed using the MERN stack. The document describes the system requirements, architecture, database design, modules, REST APIs, security approach, testing strategy, deployment considerations and future enhancements.

PROJECT PROFILE

Project Title	Personalized Fitness Class Booking Platform
Application Type	Full-Stack Web Application
Frontend	React.js, JavaScript, React Router, CSS/Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Authentication	JWT + bcrypt
API Style	REST API
Development Tools	VS Code, npm, Git/GitHub, Postman
Target Users	Users, Trainers, Administrators
Purpose	Online fitness class discovery and booking

TABLE OF CONTENTS

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Scope
- 6. Existing System
- 7. Proposed System
- 8. Feasibility Study
- 9. Requirements
- 10. Technology Stack
- 11. System Architecture
- 12. Modules
- 13. User Roles
- 14. Functional Requirements
- 15. Non-Functional Requirements
- 16. Database Design
- 17. API Specification
- 18. Authentication & Authorization
- 19. UI/Page Design
- 20. Booking Workflow
- 21. Project Structure
- 22. Installation & Configuration
- 23. Testing
- 24. Security
- 25. Deployment
- 26. Advantages
- 27. Limitations
- 28. Future Enhancements
- 29. Conclusion
- 30. References

1. Abstract

The Personalized Fitness Class Booking Platform is a full-stack web application that provides a centralized solution for discovering, scheduling and booking fitness classes. The application is designed with React.js on the client side, Node.js and Express.js on the server side, and MongoDB for persistent data storage. Users can register, authenticate, browse fitness classes, search and filter programs, view trainer details, book available classes, cancel bookings and review booking history. Trainers can manage classes and schedules, while administrators can manage users, trainers, classes and bookings. JWT authentication and role-based authorization protect restricted resources. The project demonstrates a practical MERN architecture and provides a foundation that can later support payments, notifications, recommendations and mobile applications.

2. Introduction

Fitness and wellness services increasingly require convenient digital scheduling. A web-based booking platform can reduce manual work and provide users with real-time access to class information. This project addresses that need by combining a responsive React frontend with a RESTful Express/Node.js backend and MongoDB database. The design emphasizes modularity, maintainability, security and future scalability.

3. Problem Statement

Manual or semi-manual fitness booking processes can require phone calls, messages or spreadsheets. These methods may lead to outdated schedules, duplicate reservations, difficult record management and limited visibility of available seats. Users also lack a single place to compare classes and trainers. The proposed platform automates these activities through a centralized web application.

4. Objectives

The main objectives are to provide secure registration and login; display fitness classes and schedules; support class search and filtering; show trainer information; allow authenticated users to book and cancel classes; maintain booking history; prevent unauthorized access; provide trainer management functions; provide administrator controls; and persist application data in MongoDB.

5. Scope

The project covers three major roles: User, Trainer and Administrator. The user scope includes account management, class discovery and booking. The trainer scope includes profile and class management. The administrator scope includes overall management and monitoring. The initial system is intended as a web application and can be extended to payment, notifications, analytics and mobile clients.

6. Existing System

In a traditional fitness center, users may depend on staff to communicate class schedules and availability. Bookings can be recorded manually, which makes searching, updating and reporting difficult. There may also be no automatic capacity validation or centralized booking history.

7. Proposed System

The proposed system provides an online platform where class information is maintained centrally. A React interface communicates with Express REST endpoints. The server authenticates users, validates requests, applies business rules and interacts with MongoDB. The result is a more structured booking workflow with role-specific access and persistent records.

8. Feasibility Study

Technical feasibility: MERN is widely used for full-stack web development and provides the required frontend, API and database capabilities.

Operational feasibility: The system provides a simple browser-based workflow for users, trainers and administrators.

Economic feasibility: The project can be developed using open-source technologies and can be deployed using common cloud services.

Schedule feasibility: The modular design allows features to be implemented and tested independently.

9. Requirements

Hardware: A development computer with at least 4 GB RAM, modern processor and sufficient storage; internet connection for package installation and cloud database/deployment when applicable.

Software: Node.js and npm, MongoDB or MongoDB Atlas, a modern browser, VS Code or another IDE, Git and an API testing tool such as Postman.

10. Technology Stack

MongoDB: document-oriented database for users, classes, trainers and bookings.

Express.js: backend web framework used to create REST APIs and middleware.

React.js: component-based frontend framework for the user interface.

Node.js: JavaScript runtime for the server.

Mongoose: object data modeling and schema validation for MongoDB.

JWT: token-based authentication.

bcrypt: password hashing.

React Router: client-side navigation.

11. System Architecture

The system follows a layered client-server architecture. The presentation layer is the React application. The application layer consists of Express routes, controllers and middleware running on Node.js. The data layer is MongoDB accessed through Mongoose. The browser sends HTTP requests to the API; the API validates authentication and input, performs business logic and returns JSON responses.

12. Modules

Authentication: registration, login, password hashing and token verification.

User: profile, class discovery and personal bookings.

Fitness Class: class creation, listing, filtering, editing and deletion.

Trainer: trainer profiles, specialization and class management.

Booking: booking creation, capacity checking, cancellation and history.

Admin: management of users, trainers, classes and bookings.

Dashboard: role-specific summaries and activity.

13. User Roles

User: can register, log in, browse classes, view trainers, book classes, cancel eligible bookings and view history.

Trainer: can manage trainer information and assigned fitness classes and view enrolled users.

Administrator: can manage platform entities and monitor bookings and users. Role checks are performed on protected server routes.

14. Functional Requirements

FR1: User registration and login. FR2: Secure password storage. FR3: Fitness class listing. FR4: Search/filter by category, trainer or schedule. FR5: Class details. FR6: Trainer details. FR7: Authenticated booking. FR8: Capacity validation. FR9: Booking cancellation. FR10: Booking history. FR11: Trainer class management. FR12: Administrative management. FR13: Protected routes. FR14: Persistent MongoDB storage.

15. Non-Functional Requirements

The application should be responsive, usable, maintainable and secure. API responses should be structured consistently. Authentication should protect private resources. The backend should validate input and handle failures without exposing sensitive information. The architecture should permit additional modules without major redesign.

16. Database Design

The recommended MongoDB collections are:

Users: _id, name, email, passwordHash, role, phone/profile fields, createdAt, updatedAt.

Trainers: _id, name, bio, specialization, experience, contact/profile fields, createdAt, updatedAt.

FitnessClasses: _id, title, description, category, trainerId, date, startTime, duration, capacity, availableSeats, status, createdAt.

Bookings: _id, userId, classId, bookingDate, status, createdAt, updatedAt.

ObjectId references connect users, trainers, classes and bookings.

Collection	Purpose
Users	Accounts, credentials and roles
Trainers	Trainer profiles and expertise
FitnessClasses	Schedules, capacity and trainer assignment
Bookings	User reservations and status

17. API Specification

Authentication: POST /api/auth/register, POST /api/auth/login.

Users: GET /api/users/profile, PUT /api/users/profile.

Classes: GET /api/classes, GET /api/classes/:id, POST /api/classes, PUT /api/classes/:id, DELETE /api/classes/:id.

Trainers: GET /api/trainers, GET /api/trainers/:id, POST /api/trainers, PUT /api/trainers/:id.

Bookings: POST /api/bookings, GET /api/bookings/my, GET /api/bookings/:id, DELETE /api/bookings/:id.

Administrative endpoints can be added under /api/admin for management and reporting.

18. Authentication & Authorization

During registration, passwords are hashed with bcrypt. During login, the server verifies the password and issues a signed JWT. Protected requests provide the token through the configured authentication mechanism. Middleware verifies the token and attaches the authenticated user to the request. Role-based middleware restricts trainer and admin endpoints. Secrets are stored in environment variables and must not be committed to source control.

19. UI/Page Design

The frontend can contain Home, Login, Register, Classes, Class Details, Trainers, Trainer Details, Dashboard, My Bookings, Profile, Trainer Dashboard and Admin Dashboard pages. Reusable components can include Navbar, Sidebar, ClassCard, TrainerCard, SearchBar, FilterPanel, BookingForm, Modal, ProtectedRoute and Loading/Error components. The interface should be responsive across desktop, tablet and mobile devices.

20. Booking Workflow

1. User registers or logs in. 2. User browses classes. 3. User searches or filters classes. 4. User opens class details. 5. User selects Book. 6. Server validates authentication and availability. 7. Server creates the booking. 8. Class availability is updated. 9. User receives booking confirmation in the application. 10. Booking appears in My Bookings. 11. User may cancel according to configured rules.

21. Project Structure

root/

client/ — React frontend

client/src/components/ — reusable UI components

client/src/pages/ — application pages

client/src/context/ — authentication/application state

client/src/services/ — API functions

client/src/App.jsx and main.jsx — application entry points

server/ — Node/Express backend

server/config/db.js — MongoDB connection

server/models/ — User, Trainer, FitnessClass, Booking schemas

server/controllers/ — business logic

server/routes/ — API route definitions

server/middleware/ — authentication/error middleware

server/server.js — backend entry point

.env / .env.example — environment configuration

22. Installation & Configuration

Install Node.js and MongoDB or create a MongoDB Atlas database. In the server directory run **npm install**. Create a .env file containing MONGODB_URI, JWT_SECRET and PORT. Start the backend using **npm run dev** when a development script is configured, or **npm start**. In the client directory run **npm install** and **npm run dev**. Configure the frontend API base URL if required. The terminal will display the local development address.

23. Testing

Testing should include unit-level validation of important functions, API endpoint testing and end-to-end user-flow testing. Test cases should cover valid/invalid registration, login, protected routes, class listing, class creation, class updates, class deletion, full-capacity booking, successful booking, duplicate booking prevention, cancellation, role restrictions and database failure handling. Responsive testing should cover common desktop, tablet and mobile widths.

24. Security

Security measures include bcrypt password hashing, JWT verification, role-based access control, input validation, unique email constraints, duplicate-booking prevention, controlled CORS, secure environment variables and centralized error handling. Production systems should use HTTPS, rate limiting, secure cookies where appropriate, database access controls and regular dependency updates.

25. Deployment

The frontend can be deployed to a static/web hosting service and the Node/Express API can be deployed to a server or cloud application platform. MongoDB Atlas can provide a managed database. Production configuration should define environment variables for the database URL, JWT secret, allowed frontend origin and port. After deployment, API connectivity, CORS, authentication and database access must be verified.

26. Advantages

The platform centralizes fitness-class information and bookings, reduces manual scheduling, provides role-specific workflows, maintains persistent records, improves availability visibility and offers a scalable MERN architecture. The REST API also allows future mobile or third-party clients to consume the same backend.

27. Limitations

An academic implementation may not include live payments, real-time messaging, video streaming, advanced recommendation models or native mobile applications. Production deployment also requires additional operational controls such as monitoring, backups, rate limiting and stronger infrastructure security.

28. Future Enhancements

Future versions can include online payment and subscription plans, email/SMS/push notifications, calendar synchronization, trainer ratings and reviews, attendance tracking, workout progress dashboards, AI-based personalized class recommendations, nutrition planning, live classes and a dedicated mobile application.

29. Conclusion

The Personalized Fitness Class Booking Platform is a practical full-stack MERN project that demonstrates frontend development, REST API design, database modeling, authentication, authorization and booking workflows. By separating the client, server and data layers, the application remains understandable and extensible. The platform provides a strong foundation for a production-oriented fitness booking service and can be expanded with payments, analytics, notifications and recommendation features.

30. References

React.js documentation; Node.js documentation; Express.js documentation; MongoDB documentation; Mongoose documentation; JSON Web Token documentation; bcrypt documentation; MDN Web Docs; Vite documentation. These resources provide implementation guidance for the technologies used in the project.